CMPE 492

Design and Implementation of a Robust LLMOps Platform

Ali Gökçek

Advisor:

Bahri Atay Özgövde

GitHub Repository:

https://github.com/aligokcek1/CMPE492-Design-and-Implementation-of-a-Robust-LLMOps-Platform-with-SDD

June, 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1. Broad Impact

Deploying large language models in production still requires cloud credentials, infrastructure provisioning, hardware selection, endpoint management, monitoring, and teardown, tasks that slow experimentation and often depend on dedicated DevOps expertise. This project delivers a browser-based **LLMOps platform** that guides data scientists and ML engineers from Hugging Face authentication through model upload or selection, CPU/GPU cloud deployment, proxy inference, and observability-backed teardown.

The platform implements a full operational lifecycle: models are staged on the Hugging Face Hub, deployed to **Google Kubernetes Engine (GKE) Autopilot with TGI-CPU** for cost-efficient CPU serving or to **Lightning AI (LitServe + vLLM)** for managed GPU serving, exposed through a unified inference proxy, and monitored with **Prometheus and Grafana** (Time to First Token, throughput, and hardware utilization). A single Docker Compose stack runs the Streamlit dashboard, FastAPI backend, Prometheus, and Grafana for local demonstration. The complete source code, feature specifications, and deployment instructions are available in the public GitHub repository [1].

Development followed **Spec-Driven Development (SDD)** using GitHub Spec Kit [2]: feature specifications (`spec.md`, `plan.md`, `tasks.md`) and a project constitution guided incremental delivery. SDD is the *engineering methodology* for this codebase, not a separate research deliverable.

1.2. Ethical Considerations

The platform handles sensitive credentials (Hugging Face tokens, GCP service-account keys, Lightning AI API keys). Tokens remain server-side in process-local sessions; cloud keys are Fernet-encrypted in SQLite and never returned in API responses. Hugging Face tokens are injected into deployment environments only as transient runtime variables and are not persisted in the database or logs.

As AI-assisted development becomes common, practitioners must retain accountability when generated infrastructure code is deployed to real cloud accounts. This project mitigates that risk through contract tests, fake cloud providers in CI, optional Kubernetes manifest dry-run validation, and human review of orchestration logic before production use.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

The primary deliverable is an **enterprise-oriented LLM deployment pipeline** with the following capabilities:

- Hugging Face authentication and session continuity across browser restarts.
- Robust upload of local model directories to user-owned Hugging Face repositories (resumable uploads, deploy shortcut).
- Selection and deployment of public, private, or gated Hugging Face models.
- Real cloud deployment on **CPU** (per-deployment GCP project + GKE Autopilot + TGI-CPU) or **GPU** (Lightning AI managed cloud + LitServe + vLLM).
- In-dashboard inference through a backend proxy with Prometheus metrics (TTFT, token throughput).
- Per-deployment Grafana dashboards and HMAC-signed deep links.
- Secure credential management and deployment lifecycle control (create, monitor, delete, decommission).

Incremental features were specified and implemented as numbered specs (004–011) under the repository `specs/` directory, each with acceptance tests before merge.

2.2. Project Planning

The roadmap evolved from an upload-focused MVP to a production-style LLMOps stack. Table 2.1 summarizes the delivered milestones.

Table 2.1. Delivered project milestones

Milestone	**Week(s)**	**Key Deliverable**
Foundation & SDD tooling	1–5	Constitution, Spec Kit workflow, HF auth research
Upload & session MVP	6–8	Local-to-HF pipeline, browser session persistence
GKE CPU deployment	9–12	Real GCP/GKE orchestration, inference proxy
GPU path (Lightning AI)	12–14	Hardware selector, LitServe+vLLM jobs
HF cloud deploy & shortcuts	14–15	User-uploaded models, `model_origin`, HF_TOKEN injection
Monitoring & dashboard	15–17	Prometheus/Grafana, Streamlit ops refactor

2.2.1. Project Time and Resource Estimation

2.2.1.1. Timeline Estimation. The project ran over 17 weeks. Early weeks combined inference-engine research (`vLLM`, TGI) with Spec Kit onboarding. The largest engineering effort shifted from mock deployments to real GKE orchestration (project creation, billing attachment, Autopilot cluster bring-up, manifest apply) and subsequently to the Lightning AI GPU path and observability stack.

2.2.1.2. Resource Estimation. **Hardware and cloud:** CPU deployments consume GCP projects and GKE Autopilot clusters per deployment; GPU deployments use Lightning AI managed infrastructure. Local development uses Docker Compose with optional `LLMOPS_USE_FAKE_GCP=1` for demos without real GCP calls.

Software stack: Python 3.11, FastAPI, Streamlit, SQLAlchemy/SQLite, Google Cloud and Kubernetes clients, Prometheus v2.55, and Grafana 11.4. Key Python packages include `huggingface_hub`, `lightning-sdk`, and `prometheus-client`.

Financial: Free 300$ GCP credits covered CPU-path integration testing; GPU experiments used 15$ Lightning AI student credits matching the contingency identified in early risk analysis.

2.2.2. Success Criteria

Success is measured primarily by **functional pipeline outcomes**:

- End-to-end Hugging Face staging with file integrity and retry-capable uploads (up to 10 GB per file in Streamlit, configurable backend total limit).
- Automated CPU deployment to GKE with a reachable OpenAI-compatible endpoint and correct teardown on delete.
- Automated GPU deployment to Lightning AI with endpoint polling until `running`.
- Proxy inference records TTFT and token metrics; Prometheus scrapes and Grafana dashboards provision per deployment.
- Contract and integration tests pass in CI with fake cloud providers; no secrets in logs or API responses.

SDD supported delivery quality (constitution-gated TDD, spec-to-task traceability) but is not evaluated as a standalone empirical study in this report.

2.2.3. Risk Analysis

Risk	**Potential Impact**	**Mitigation (Outcome)**
Cloud credit exhaustion	Cannot benchmark CPU/GKE paths.	Fake GCP in CI; optional fake mode for demos; Lightning AI for GPU.
GCP GPU quota / cost	No affordable GPU on GKE.	GPU path routed to Lightning AI instead of GKE L4.
Long GKE bring-up	Failed deploys discard 15–25 min of work.	Orchestrator retains cluster on downstream failure; user-driven delete tears down project.

Model architecture mismatch	CPU deploys use Hugging Face TGI-CPU, which only serves compatible `text-generation` / NLP checkpoints on the Hub; GPT-style, multimodal, or non–causal-LM architectures (e.g., models without a supported TGI loader) fail at preflight or crash at runtime even if the repo exists.	Pre-deploy `pipeline_tag` gate returns `unsupported_model`; UI directs users to GPU (Lightning AI + vLLM) for broader serving; user-owned uploads may bypass missing tags but remain operator responsibility.
Credential leakage	HF or cloud keys exposed.	Server-side sessions; Fernet encryption; transient HF_TOKEN only.
Agent-generated IaC errors	Broken manifests or cloud calls.	TDD, contract tests, optional K8s dry-run suite.
Scope creep	Fine-tuning distracts from core pipeline.	Deferred; core LLMOps lifecycle completed first.

3. RELATED WORK

3.1. High-Performance LLM Serving and Inference

The KV cache dominates memory use in LLM serving. Kwon et al. [3] introduced PagedAttention in vLLM [4], improving throughput by reducing KV-cache fragmentation. This platform uses **TGI-CPU** on GKE for the CPU path and **vLLM via LitServe** on Lightning AI for the GPU path, reflecting a split between cost-efficient CPU serving and managed GPU inference.

3.2. Spec-Driven Development

SDD treats specifications as executable artifacts that guide AI-assisted implementation [5]. GitHub Spec Kit [2] provides constitution, specify, plan, task, and implement commands. In this project, SDD reduced unstructured "vibe-coding" by anchoring each feature branch to `spec.md` and contract tests; it complemented rather than replaced conventional code review and cloud validation.

3.3. Agentic Workflows and Industry Practice

Structured system instructions (e.g., Spotify's HONK [6]) and explicit negative constraints improve reliability of generated code. The project constitution plays a similar role: security-first handling of tokens, direct library usage, and mandatory TDD before feature completion.

4. METHODOLOGY

4.1. Application Development Workflow

The platform follows a conventional three-tier pattern: Streamlit frontend, FastAPI backend, SQLite persistence, and external integrations (Hugging Face Hub, GCP/GKE, Lightning AI, Prometheus/Grafana). User operations map to REST endpoints; long-running deploy and delete operations run asynchronously in a `DeploymentOrchestrator` state machine (`queued` → `deploying` → `running` / `failed` / `lost` / `deleted`).

4.2. Spec-Driven Development as Engineering Practice

Features were developed with GitHub Spec Kit using four recurring artifacts:

(i) **Constitution** (`.specify/memory/constitution.md`): global principles (security, TDD, simplicity).
(ii) **Specification** (`spec.md`): user stories and functional requirements per feature.
(iii) **Plan** (`plan.md`): architecture, dependencies, and file-level design.
(iv) **Tasks** (`tasks.md`): ordered, testable implementation steps.

When behavior drifted, corrections targeted the specification and tests first, then the implementation. Eight incremental specs (004–011) delivered upload, sessions, GKE CPU deploy, GPU deploy, HF cloud deploy, monitoring, and dashboard refactor; each merged with pytest contract coverage and Ruff linting in CI.

5. REQUIREMENTS SPECIFICATION

5.1. Functional Requirements

The final platform satisfies the following requirements (grouped by capability):

- **Authentication (FR-A):** Users authenticate with Hugging Face access tokens; the backend issues a server-side session and never stores tokens in the frontend beyond a session cookie.
- **Model intake (FR-M):** Users upload local model files or select existing Hugging Face repositories; uploads are retry-capable and may set a deploy shortcut to pre-fill the Deploy tab.
- **Credentials (FR-C):** Users store GCP service-account JSON (CPU) and Lightning AI API keys (GPU), encrypted at rest, with validation status surfaced in the UI.
- **Deployment (FR-D):** Users deploy models with explicit `cpu` or `gpu` hardware; the system preflights model support, sets `model_origin` (`uploaded` vs. `public`), and returns 202 while orchestration continues asynchronously.
- **Inference (FR-I):** Users run chat-style inference on running deployments through a backend proxy that records TTFT and token counts.
- **Observability (FR-O):** Running deployments expose native Streamlit metrics charts, Prometheus scrape jobs, per-deployment Grafana dashboards, and signed Grafana redirect links.
- **Lifecycle (FR-L):** Users delete deployments; cloud resources are torn down and monitoring is scheduled for decommission (7-day operator retention).
- **Reliability (FR-R):** Upload and deploy operations support idempotency keys; invalid cloud credentials block new deploys but do not stop running workloads.

5.2. User Scenarios

(i) **Sign in with Hugging Face (P1):** Authenticate to access upload, deploy, and inference features securely.

(ii) **Upload and stage local models (P1):** Upload a local LLM directory to a Hugging Face repository for stable cloud deployment.

(iii) **Select existing models (P2):** Pick a public, private, or gated repository without re-uploading.

(iv) **Deploy to cloud with CPU or GPU (P1):** Provision real infrastructure and receive a live inference endpoint.

(v) **Run inference and view metrics (P1):** Test the deployment in-app and inspect TTFT, throughput, and hardware charts.

(vi) **Delete and decommission (P2):** Tear down cloud resources and release monitoring artifacts on schedule.

5.3. Use Case Diagram

Figure 5.1 maps platform use cases to the user and to external systems: Hugging Face Hub, GCP, Lightning AI, and Prometheus/Grafana.

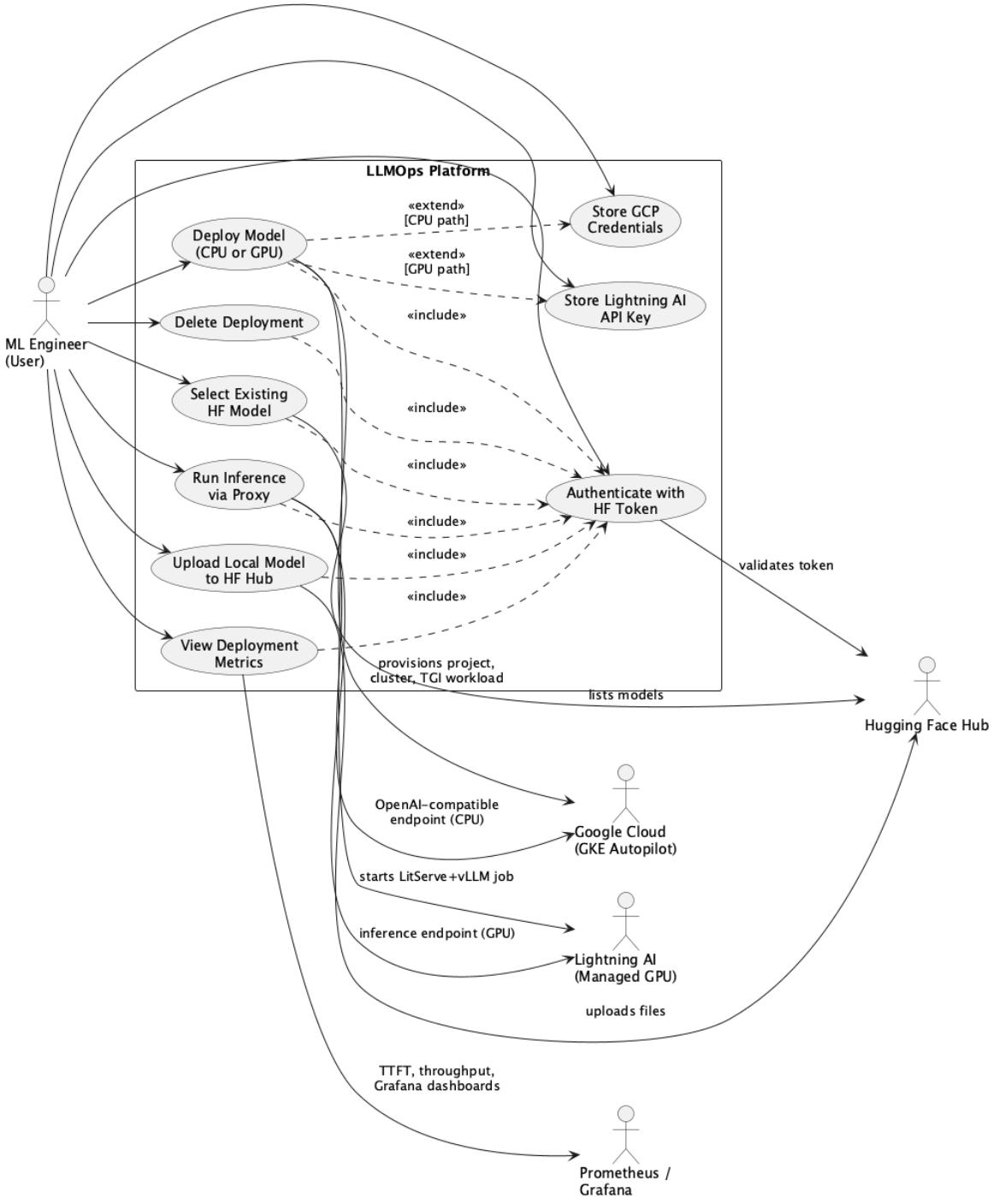

Figure 5.1. Use case diagram (CPU/GPU deploy, inference, metrics).

6. DESIGN

6.1. Information Structure

Persistent state lives in SQLite (`backend/data/llmops.db`): encrypted GCP and Lightning AI credentials, deployment rows (hardware-specific fields), and deployment monitoring metadata. Hugging Face tokens remain in the in-memory session store only. Figure 6.1 shows entity relationships.

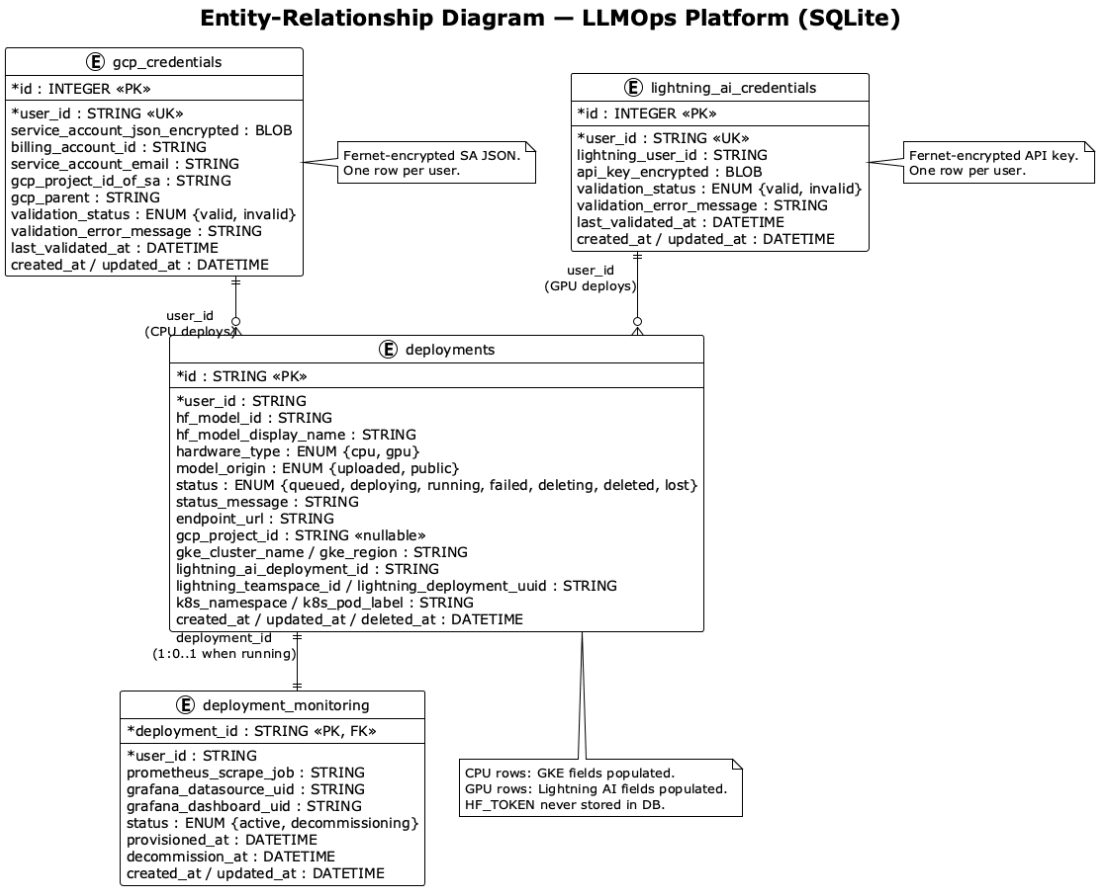

Figure 6.1. Entity-relationship diagram (SQLite schema).

6.2. Information Flow

6.2.1. Activity Diagram

Figure 6.2 covers authentication, optional upload, credential setup, hardware-specific deploy, monitoring provision, inference, and deletion.

6.2.2. Sequence Diagram

Figure 6.3 details shared authentication and deploy entry, `alt` branches for GKE vs. Lightning AI orchestration, and shared inference/metrics flows.

6.3. System Design

The system splits into a Streamlit operations dashboard and a FastAPI backend (Figure 6.4). Routers cover auth, upload, models, deployments, metrics, and credential endpoints. Services encapsulate orchestration, Hugging Face integration, manifest generation, inference proxy, and monitoring provisioners.

6.3.1. Class Diagram

Figure 6.5 summarizes key stores, orchestrators, provider interfaces, and API client boundaries.

6.4. User Interface Design

The Streamlit dashboard uses four main tabs (**Deployments**, **Upload Model**, **Select Model**, and **Deploy**) plus a sidebar **Settings** panel for GCP and Lightning AI credentials. Invalid credentials trigger persistent warning banners; running deployments are unaffected.

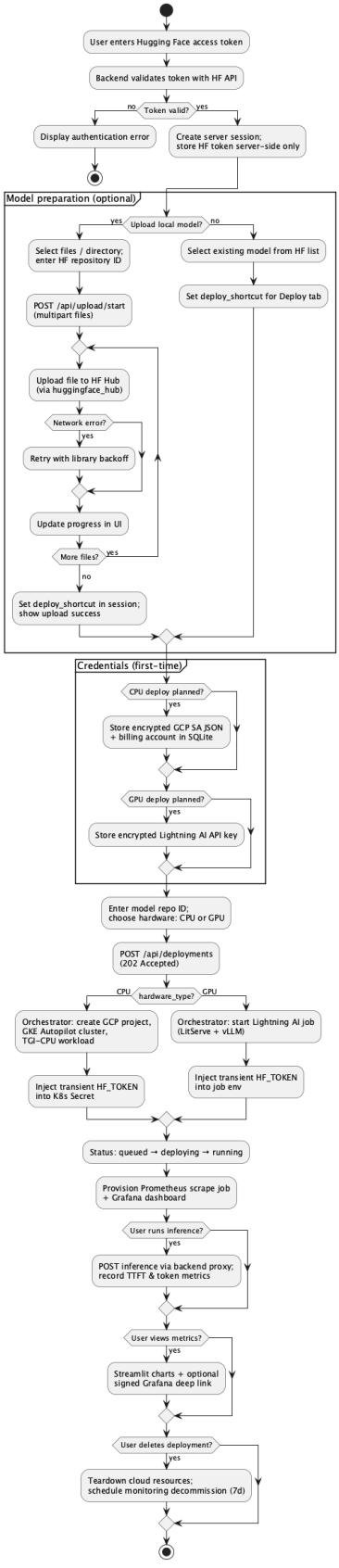

Figure 6.2. Activity diagram: upload, deploy (CPU/GPU), inference, and teardown.

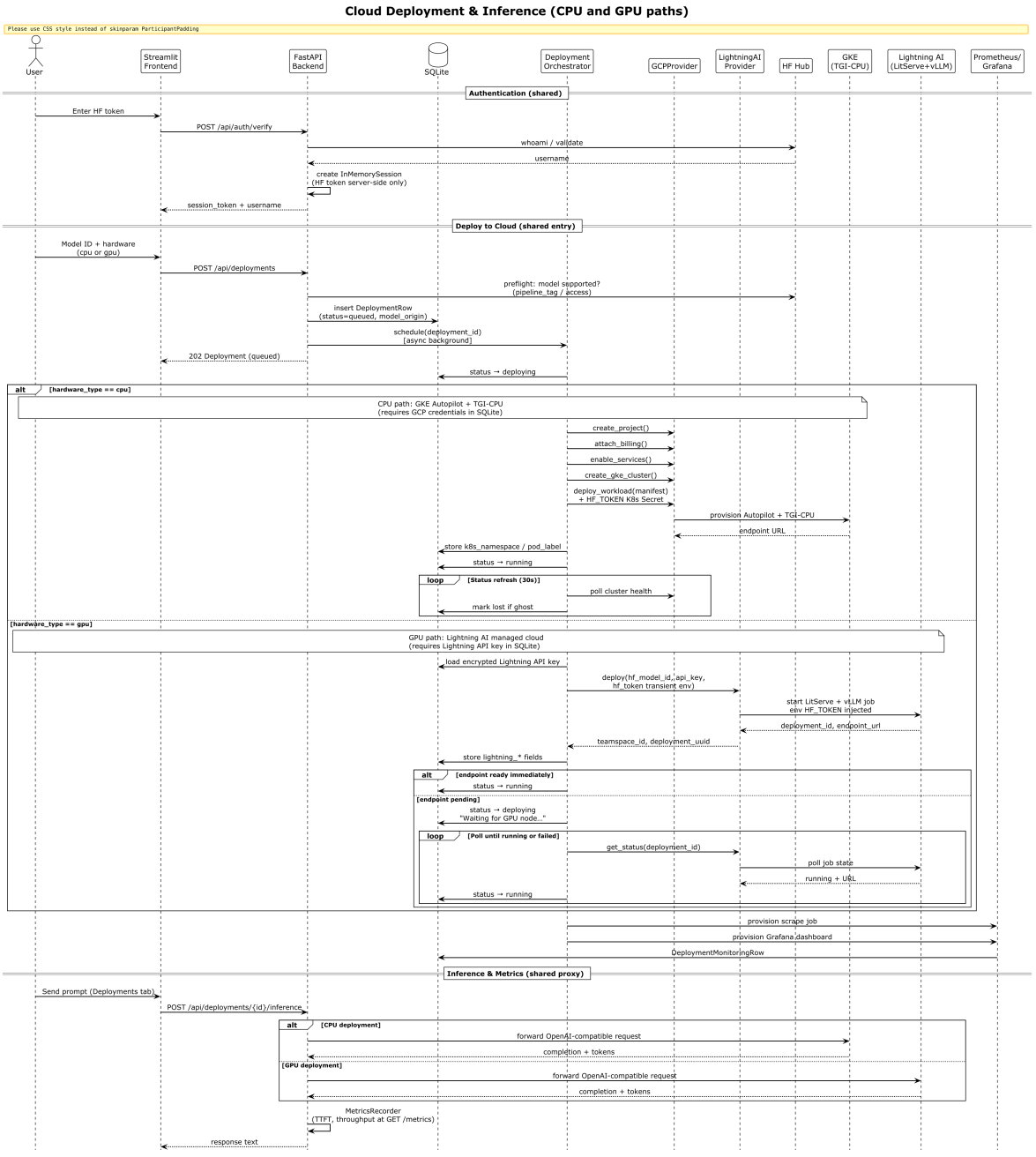

Figure 6.3. Sequence diagram: authentication, CPU/GPU deployment, inference, and metrics.

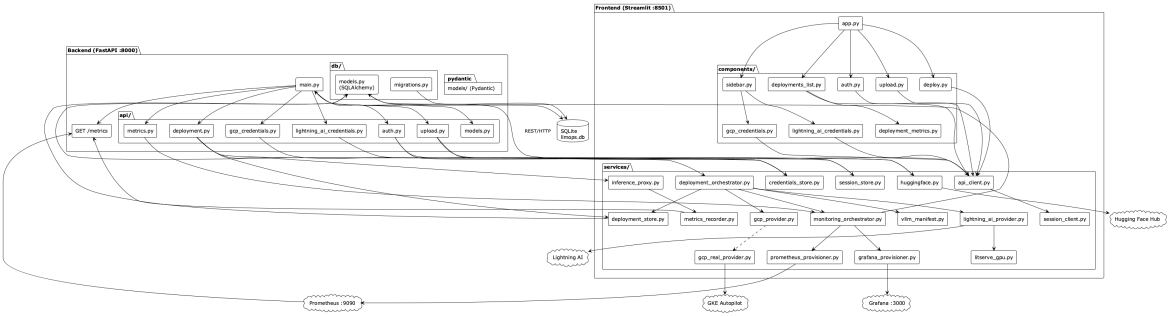

Figure 6.4. Module diagram: frontend components, API routers, services, and external systems.

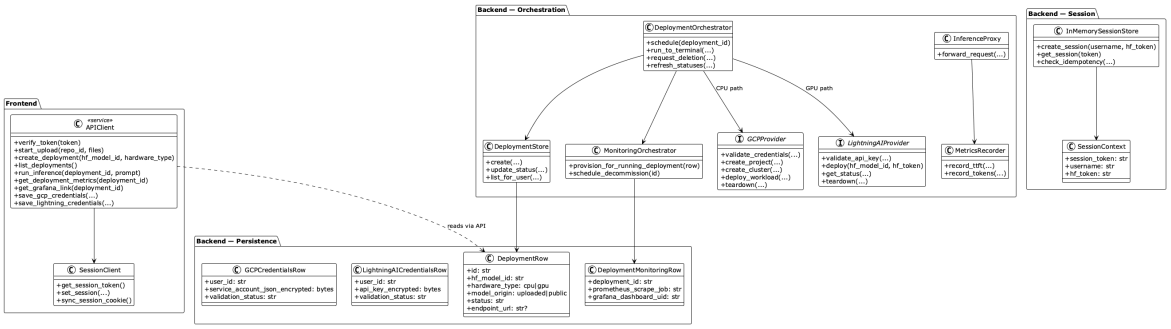

Figure 6.5. Class diagram: orchestration, providers, persistence, and API client.

The **Deployments** tab lists active workloads with status, hardware type, and shortcuts to inference and metrics (Figure 6.6). The **Deploy** tab accepts a Hugging Face repository ID, fetches model metadata, and exposes an explicit CPU/GPU selector; the action label reflects the target provider (GKE for CPU, Lightning AI for GPU), as shown in Figure 6.7. Once a deployment is `running`, users run chat-style inference in-app with adjustable temperature and token limits (Figure 6.8). Metrics appear as native Streamlit charts (throughput, CPU, and GPU utilization) and, optionally, via an **Open in Grafana** deep link for operator-grade dashboards (Figures 6.9 and 6.10).

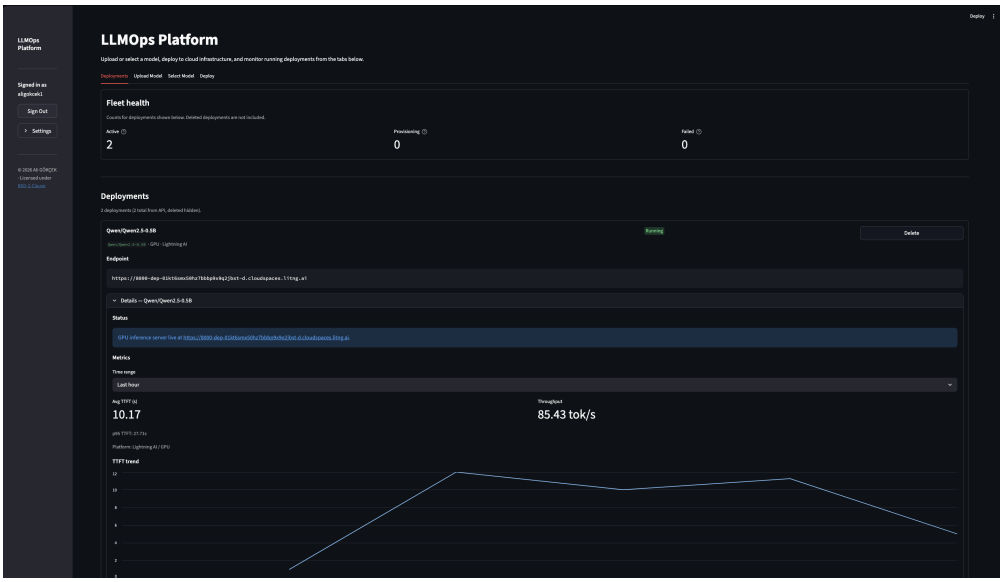

Figure 6.6. Deployments tab: fleet overview with status, hardware, and entry points to inference and metrics.

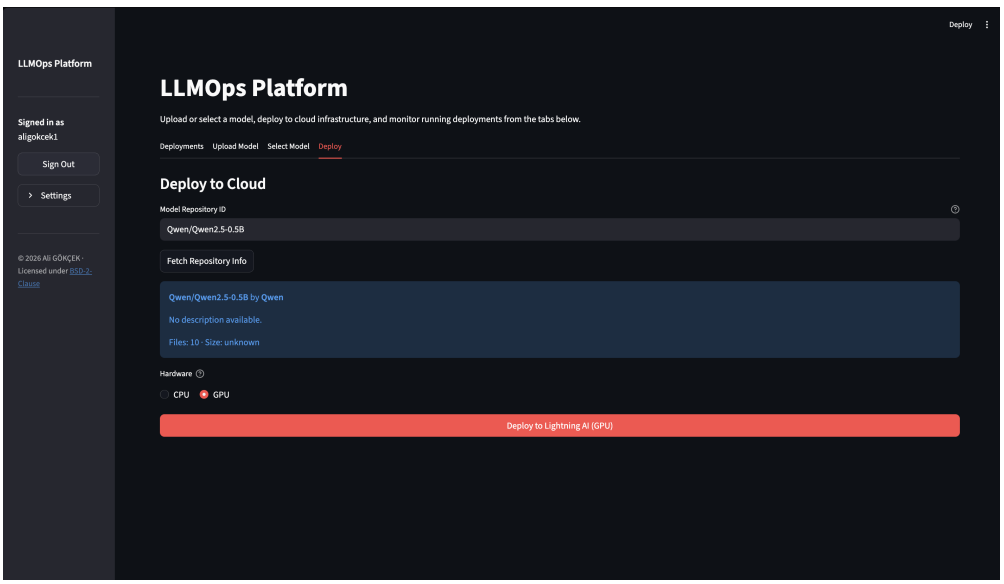

Figure 6.7. Deploy tab: repository preflight, GPU hardware selection, and Lightning AI deploy action.

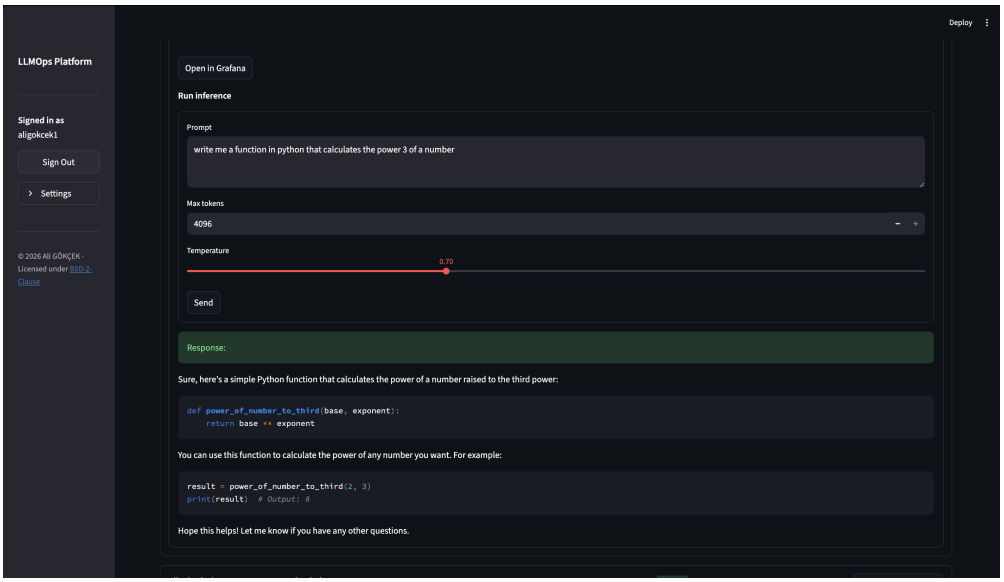

Figure 6.8. In-dashboard inference proxy with prompt, generation parameters, and model response.

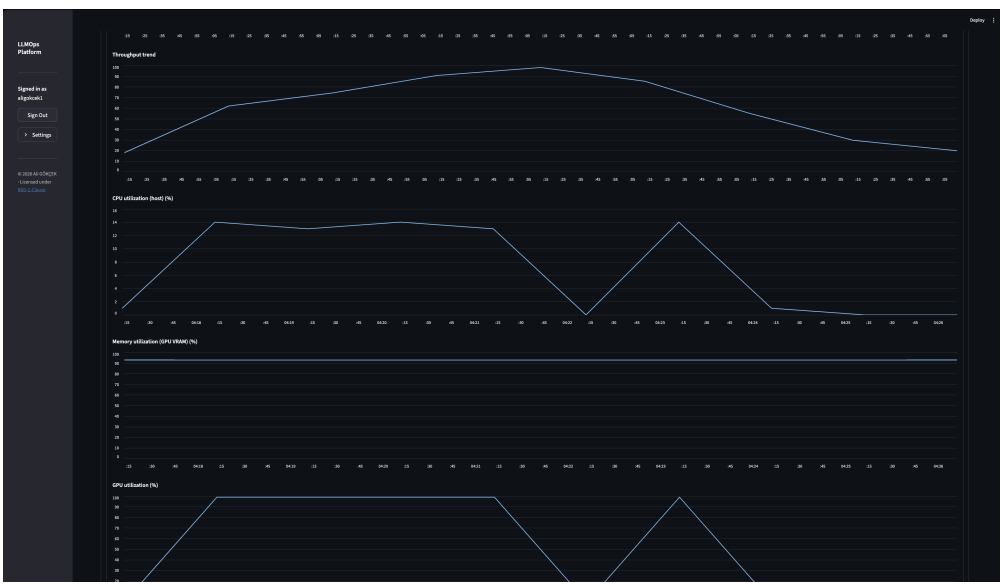

Figure 6.9. Native metrics panel: throughput trend and CPU/GPU utilization time series.

Figure 6.10. Per-deployment Grafana dashboard opened via signed redirect from the platform.

7. IMPLEMENTATION AND TESTING

7.1. Implementation Overview

Backend (FastAPI): Registers auth, upload, models, deployment, metrics, GCP, and Lightning AI routers; mounts Prometheus metrics at `GET /metrics`; runs background tasks for deployment status refresh, hardware metric polling, and monitoring decommission.

Frontend (Streamlit): Tab-based workflow with `api_client` and cookie-backed `session_client` for continuity.

CPU path: `DeploymentOrchestrator` creates a dedicated GCP project, attaches billing, enables services, provisions GKE Autopilot, applies a TGI-CPU manifest (`vllm_manifest.py`) with a Kubernetes Secret for the session HF token, and stores workload labels for metrics polling.

GPU path: The orchestrator calls `LightningAIProvider.deploy()` with a generated LitServe+vLLM script (`litserve_gpu.py`), polls until an endpoint is ready, and persists Lightning deployment identifiers.

Monitoring: `MonitoringOrchestrator` writes Prometheus scrape jobs and provisions Grafana dashboards; `InferenceProxy` records TTFT and token counters consumed by PromQL queries and Streamlit charts.

Security: Fernet encryption via `LLMOPS_ENCRYPTION_KEY`; HMAC-signed Grafana URLs via `LLMOPS_GRAFANA_SIGNING_SECRET`; `model_origin` and pipeline-tag rules for user-uploaded models.

7.2. Deployment

This section describes how the platform itself is deployed for local demonstration. The runtime layout of the local stack follows the architecture in Figure 6.4: a Streamlit frontend, FastAPI backend, Prometheus, and Grafana orchestrated by Docker Compose.

7.2.1. Local Platform Stack (Docker Compose)

The recommended entry point is the repository root `README.md` in the GitHub project [1]. From the project root, a single command builds and starts all platform services:

```
docker compose up -d --build
```

On first run, Docker volumes persist generated secrets (`LLMOPS_ENCRYPTION_KEY` and `LLMOPS_GRAFANA_SIGNING_SECRET`) and the SQLite database under `backend/data/llmops.db`; no manual `.env` file is required for a default demo. Table 7.1 lists the exposed endpoints.

Table 7.1. Local Docker Compose services

Service	**URL**	**Role**
Streamlit dashboard	`http://localhost:8501`	User interface
FastAPI backend	`http://localhost:8000`	REST API, orchestration
Prometheus	`http://localhost:9090`	Metrics storage (30-day retention)
Grafana	`http://localhost:3000`	Dashboards (`admin`/`admin`)

To stop the stack, run `docker compose down`. To reset credentials and the

database, use `docker compose down -v`. For demonstrations without billing a real GCP account, prefix the compose command with `LLMOPS_USE_FAKE_GCP=1` (see `README.md`) or add it under `backend.environment` in `docker-compose.yml`.

7.2.2. Native Development Deployment

Developers may run components separately (three terminals, as in `README.md`):

- **Monitoring:** `docker compose -f docker-compose.monitoring.yml up -d`
- **Backend:** from `backend/`, export `LLMOPS_ENCRYPTION_KEY`, then run `uvicorn src.main:app --reload`
- **Frontend:** `streamlit run src/app.py` (from `frontend/`)

This path is intended for active development and pytest, not for operator handoff.

7.3. Testing

Development followed constitution-mandated **Test-Driven Development**: contract tests define API behavior first; implementation follows. The test pyramid includes:

- **Contract tests** under `backend/tests/contract/`: auth, upload, deployment, metrics, and credentials via `FakeGCPProvider` and `FakeLightningAIProvider` (no real cloud calls).
- **Frontend integration tests**: Streamlit workflow scenarios including hardware selection, deploy shortcuts, and badge display.
- **Optional dry-run tests**: Kubernetes manifest validation against a real API server with `dry_run=["All"]` when `LLMOPS_K8S_DRYRUN_KUBECONFIG` is set.

CI runs pytest and Ruff on backend and frontend and builds Docker images for smoke validation.

8. RESULTS

8.1. Functional Outcomes

The delivered platform meets the success criteria defined in planning:

- **Authentication:** Hugging Face token verification and server-side sessions work reliably; session cookies restore dashboard state after browser refresh.
- **Staging:** Large directory uploads to Hugging Face succeed with library-backed retry; deploy shortcuts connect upload and deploy tabs.
- **CPU deployment:** GKE Autopilot deployments reach `running` with OpenAI-compatible endpoints; delete tears down GCP projects.
- **GPU deployment:** Lightning AI jobs start with LitServe+vLLM; endpoints become available after platform polling.
- **Inference and metrics:** Proxy inference returns completions and increments Prometheus counters; Streamlit and Grafana surfaces show TTFT, throughput, and hardware series where available.
- **CI safety:** Full contract suite passes with fake providers; no HF tokens appear in database columns or test assertions for secrets.

8.2. Development Methodology

Spec Kit structured incremental delivery: each feature branch retained traceability from user story to contract test. The constitution's security and TDD principles prevented token persistence in the frontend and enforced fake cloud boundaries in automated tests. SDD accelerated feature scaffolding but still required manual validation against real GKE and Lightning AI behavior during integration.

9. CONCLUSION

This project implemented a **robust LLMOps platform** that takes Hugging Face models through upload or selection, real CPU/GPU cloud deployment, proxy inference, Prometheus/Grafana monitoring, and controlled teardown from a single operations dashboard. The dual-path architecture (GKE for CPU, Lightning AI for GPU) addresses cost and quota constraints while keeping one API and UI for users.

Spec-Driven Development with GitHub Spec Kit was used as a **development methodology** to govern quality and incremental delivery; the primary contribution is the operational pipeline itself, not a comparative study of SDD tools.

9.1. Future Work

- Multi-user tenancy and role-based access beyond per-session isolation.
- Configurable infrastructure (cloud regions, machine profiles, and default provider settings).
- Support for all model architectures beyond the current TGI-CPU and vLLM serving constraints.
- Alerts for live deployments (latency spikes, inference errors, and resource saturation).
- Optional fine-tuning or evaluation jobs integrated into the same lifecycle.
- Extended metrics retention beyond the current 30-day Prometheus window and 7-day post-delete monitoring retention.

REFERENCES

1. Gökçek, A., "CMPE492: Design and Implementation of a Robust LLMOps Platform with SDD", `https://github.com/aligokcek1/CMPE492-Design-and-Implementation-of-a-Robust-LLMOps-Platform-with-SDD`, 2026.

2. GitHub, "Spec-kit: Spec-Driven Development Toolkit", `https://github.com/github/spec-kit`, 2025.

3. Kwon, W., Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention", *arXiv preprint arXiv:2309.06180*, 2023.

4. vLLM Team, "vLLM: A High-Throughput and Memory-Efficient Inference and Serving Engine for LLMs", `https://github.com/vllm-project/vllm`, 2025.

5. Sogl, D., "Spec-Driven Development (SDD): The Evolution Beyond Vibe Coding", Medium Article, `https://danielsogl.medium.com/spec-driven-development-sdd-the-evolution-beyond-vibe-coding-1e431ae7d47b`, 2025.

6. Spotify Engineering, "Background Coding Agents: Context Engineering (Honk, Part 2)", `https://engineering.atspotify.com/2025/11/context-engineering-background-coding-agents-part-2`, 2025, Spotify internal background coding agent codename: Honk.

APPENDIX A: PROJECT CONSTITUTION (SUMMARY)

The full constitution lives at `.specify/memory/constitution.md` (v1.0.0). Core principles:

(i) Clean, self-explanatory code with minimal comments.

(ii) Security first: no credential or token leakage.

(iii) Direct framework and library usage without redundant wrappers.

(iv) Mandatory TDD (red-green-refactor) before marking tasks complete.

(v) Realistic testing: contract tests and fake providers in CI; prove features work.

(vi) Simplicity: minimal diffs; fix root causes.

APPENDIX B: FEATURE SPECIFICATION INDEX

Formal specifications for delivered capabilities are versioned under `specs/`:

- **004–006:** Model upload, session persistence, early deploy flows.
- **007:** GKE CPU inference pipeline and SQLite credential store.
- **008:** GPU/CPU hardware selector and Lightning AI credentials.
- **009:** Cloud deployment of user-uploaded Hugging Face models.
- **010:** Prometheus/Grafana per-deployment monitoring.
- **011:** Streamlit operations dashboard refactor.

Each directory contains `spec.md`, `plan.md`, and `tasks.md` as the authoritative feature record.